

业务系统架构实践总结

我从2015年起至今2022年，在业务平台（汇金结算、订购、资金）、集团财务平台（应收应付、账务核算、财资、财务分析、预算）、本地生活财务平台（发票、结算、预算、核算、稽核）所经历的业务系统研发实践的一个总结。

1.

核心是面向复杂性业务支撑的实践经验（个人概念里的“复杂业务“，大概至少面向5类行业若干业务线且业态差异很大），文章不涉及性能、稳定性、资损防控、大数据离线研发，聚焦在线业务系统架构对多态业务的包容性、开放性、灵活性、可读性。

2.

文章较多强调”个人”两字，因为仅是我个人在实践上归纳总结的一些方式方法，偏野路子比较散点微观，且自己也不敢确定观点正确与否，只是作为个人总结。

3.

实践经验主要来自两类，一类是接手旧系统，得以见识不一样的设计，文中“见过”特指。二类是自己新建的，之后又一而再三重构的，这类姿势下，人的反思记忆尤为深刻。

4.

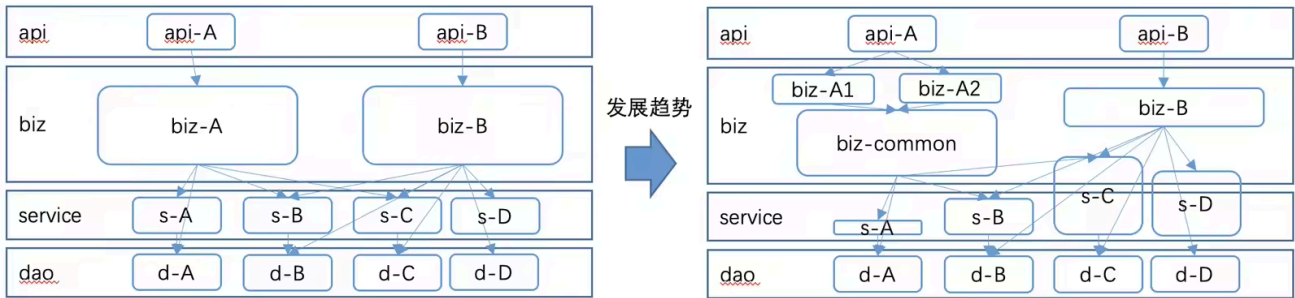
全文中，我会拿实践中的具体案例说明，但最早的要追溯到2015年了，只能依稀记得大概，可能有不全乃至不正确之处。有些案例是曾经的兄弟团队的，假使阐述有误，先道声歉，钉我立改。

单系统内架构形态（反面）

下面是个人经历过的反面案例

1)

业务层臃肿，能力层单薄



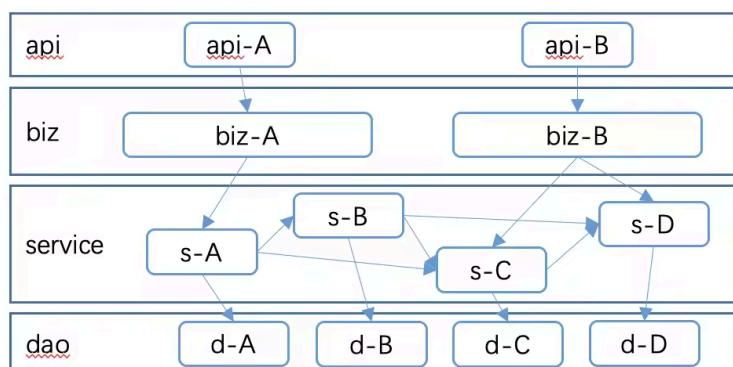
这类形态挺常见的，最初设计时做四层划分，也比较清晰。最核心的设计点是，biz层编排“可复用的”service层，完成一个场景逻辑表达。问题在于，第一，service本身的划分、定位，相对随意，从第一

感觉而来，并未经过领域划分这样的设计。所以这里特意叫service，从而区别于domain的表达，我采用的领域划分方法下文会阐述。第二，service本身不可扩展，多态业务冲击下，为适配此service能力而存在的个性向共性的转换逻辑上浮。因为这些问题的存在，随更多业务接入的架构演进下，组织扩大后的人员差异扩大下，会导致几个问题：1、biz层越发的膨胀，service层越发的萎缩。biz层里充斥了各种本该往下沉淀的可复用业务逻辑，service层则几乎萎缩为dao。2、人员的差异下，service实例颗粒度不一（有萎缩有膨胀），有重复相似的。biz实例亦是如此。3、由于biz膨胀，层内会发展为两小层，上小层为面向单一业务场景的“业务biz层”，下小层为通用可复用场景的“通用biz层”，且这两层隐约存在，随研发个体认知差异或隐或现。由于service的萎缩，biz层会干脆直接调用dao，且因人而异的随机性差异，导致biz往下的调用关系呈乱麻态。

这个形态，我印象最深的是2015年在汇金结算研发时，经历过。当时设计理念是，biz层通过hjbpm编排底层可复用的service，表达业务逻辑，甚至试图以可视化界面让业务运营自编排，当时的设计理念和初衷都是很好的。在经历大规模多态业务接入的冲击下（最初的天猫佣金结算，发展到近一半集团业务的结算），biz层产生了严重的膨胀，导致最复杂最难懂最易出问题的逻辑都堆积在biz层。我们大概在2016年做了分层、分域、SPI式开放化架构升级。

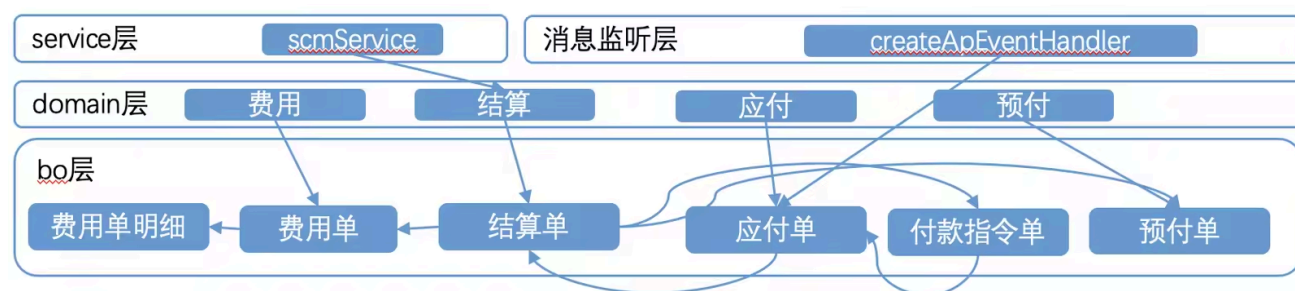
2)

service间网状调用



这类形态也挺常见的。同样，核心的问题还是对service这层的颗粒度、职责定位不清晰，对增量service的架构监管不足，业务压力下（祸起于常见的倒排需求，常以需求完整度妥协+架构方案妥协追赶deadline死线，挖下损害长期利益的坑），一线研发同学就容易凭感觉去新增service。与上文形态1)不同的是，该组研发未明确和共识biz层的“编排”作用，基于原本bizA -> serviceA 的实现链路下，随新增业务逻辑新起的serviceB（事是A的事，但不合适放serviceA，所以新建），链路演变成了bizA -> serviceA -> serviceB。这样的趋势持续发展下去，会发现bizA下的service调用链路越发的复杂，呈现为一颗深度调用树，而biz层失去了业务编排的作用退化为一个业务场景入口的标志符。从代码阅读层面讲，以为是做A（因为bizA -> serviceA的起手势），实则带着一串B+C+D+E等等，且越挖越深，不见尽头。一个新同学除非步履维艰的推演完全串调用代码，否则不能知晓bizA业务场景真实完全的意义。步履维艰在于，这是一颗深度调用树，当你阅读到serviceE时，很可能你已经忘记了最初是从哪个biz入口看过来的，以及接下来你将还会遇到哪些service讲述剩余的业务逻辑。而一个写的并不那么好的代码，还常将这些service的调用关系，隐藏在极其隐晦的角落，给阅读者增加复杂。譬如通过一个叫util的类，完成了service B -> C这一主干链路的传递。一个service的改动，几乎

无法评估其产生的影响面。而因为无法评估，又怕改出故障，在业务需求压力下，会出现各类fork行为加剧加速腐化。这是恶性循环。



2019年经历过集团智付系统的形态，类似。智付大致的层级为service(应用服务层)->domain (领域服务)->bo(业务对象操作)->ability (能力) ->extension (扩展点)。核心逻辑都集中在BO这层。它以模型为单位表达逻辑操作，譬如ExpenseBO (费用单), ExpenseDetailBO(费用单明细), SettlementBO(结算单)。

第一，domain很薄，譬如SettlementDomain，直接将函数请求传递给SettlementBO做具体处理，自己没什么逻辑。而且上层的service、eventHandler也可以穿过domain直接调用BO层。Ability很薄，只做TMF框架里对extension扩展点的调用，没自己的业务逻辑。这个点上讲，domain、ability两层有强套TMF架构的意思。当然，从实操上讲，TMF2.0架构个人认为略重了点，层次和概念有点多，譬如像activity、ability这样的概念其实可以简化掉。仅个人观点，这里不展开讲。

第二，BO以单个模型组织业务代码，切太细，譬如费用单BO、费用单明细BO、结算单BO、结算单扩展信息BO、结算单扣减记录BO、结算单扣减记录明细BO，整个应用有120个BO（截止2022年8月重新数了遍）。BO试图基于单体模型表达逻辑，但又不是充血模型的设计，仅是把纯过程式逻辑表达函数从语义关联性归类到模型。个人理解，它本质上并不是一个业务模型的表达，而是业务逻辑分组安放的壳（譬如ExpenseBO.createInvoiceBill()，因为基于费用生成收票账单，所以createInvoiceBill函数挂载到了费用BO，但事实上操作的是收票账单模型）。当然，这个设计有智付业务特性的表达，因为智付最核心的逻辑是从费用单、结算单、进项票、应付单、打款单的逐步推进，都是针对模型的加工和传递，以BusinessObject业务模型为载体组织代码很自然。但切太细，一个BO无法独立完成一个业务语义的操作，需要协同其它BO才能完成。譬如，ExpenseBO.createExpense()创建费用单，需要调用ExpenseDetailBO.createExpenseDetail()构造和持久化费用单明细。

第三，BO间跨域调用。譬如，SettlementBO.createSettlement()，创建结算单，需要调用ExpenseBO.updateExpenseToSettled() 回写费用单状态为已结算，跨了结算、费用两个域。譬如，AccountPayableBO.createAccountPayable() 创建应付单，内部调用SettlementBO.writeOffPpmt()通过对应的结算单找核销关系，做应付单核销；内部调用PrepaymentBO.unfrozenWriteoffAmount()解冻预付单,并调用PaymentInstructionBO.writeoff()做应付和预付核销，内部调用AccountPayableBO.changeInstructionAmount(), AccountPayableBO.updateStatus(), 跨了应付、结算、预付三个域。

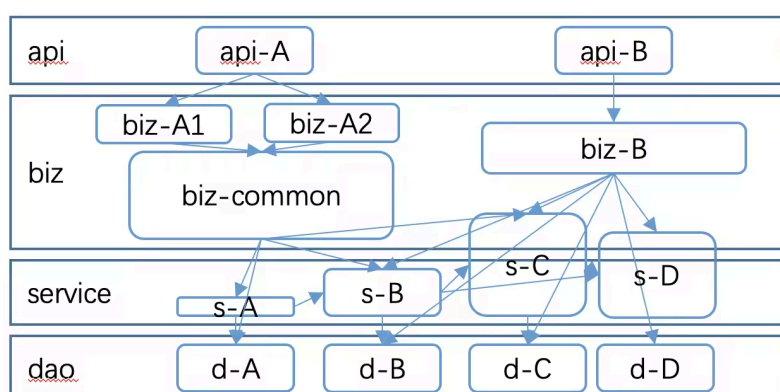
第四，event串接流程。除了上图直接函数调用的情况，智付还大量通过内部event桥接各操作节点。譬如费用单创建后，发送expense_auto_settle消息。对应消息handler调用SettlementBO创建结算单，并发送settlement_creation消息。对应消息handler调用AccountPayableRequestBO创建应付创

建请求单，并发送create_ap消息。对应消息handler调用AccountPayableBO创建应付单。等等不详述。主流程中间还穿插着net抵扣、无票税额抵扣、预付抵扣等分支流程。这种event的传递方式，相较于上层业务场景层一个函数体内过程式代码编排的方式，从代码阅读和梳理的角度，不够直观。

可以看到，BO间呈复杂庞大的网状调用链，像树根一样盘根错节潜藏在下层。且调用传递方式不一，有BO间跨域函数调用、event传递调用、甚至BO跨域反调上层service再由后者逐个调用domain、BO完成。智付平台属性下业务空间大，包含的模型、事件数量庞大，形成了一个巨大复杂的关系网，阅读其代码，容易迷失在深度调用栈中忘记了自何而来身处何处。

3)

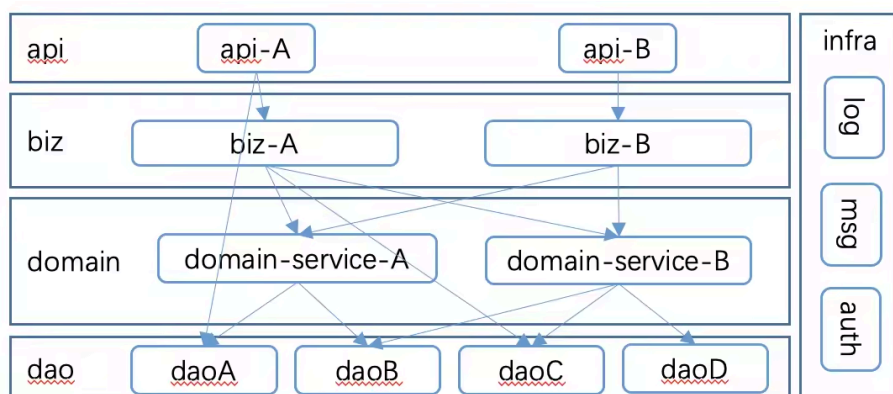
混合态



真实世界里，1、2两种形态会同时发生，事实上都是混合态，只是问题偏重度不同，分开阐述便于理解。

单系统内架构形态（采用）

个人在实践里采用的形态



看起来似乎和前面列的案例没大区别，但在实践结果上是有差异的。个人认为核心2个点：

1.

起步时的domain设计：有理论支撑的domain划分和凭经验感觉划分的service，对整个架构生命力的影响是完全不同的

2.

过程中的架构原则：必须有几个易记牢记并坚持贯彻的架构原则（见下文的实践细节），在实操层面上，对整个架构性命的维持是完全不同的

细项1 - 分层

4层定位：

1.

api层，我比较更喜欢叫应用服务层（因为API其实是个协议范畴的语义，非实现），面向应用层面对外的服务表达。灵活为主，薄，可随业务、渠道定义独立API。

2.

biz层，业务层，面向一种业务场景的逻辑表达，如下单。负责业务编排，一个业务场景的主流程应在这一层直观可视。灵活为主，薄，可特定场景独立biz类。

3.

domain层，领域服务层，围绕一簇模型操作的逻辑表达，例如费用单域、账单域。核心能力的所在地，要规范、要厚、沉淀复用为主，通过SPI兼顾业务个性化。

4.

dao层，存储层，面向一个存储对象操作的逻辑表达，无论是内部存储 or 外部存储（如rpc外调）。灵活为主，可随db差异、性能需求独立dao方法。

ps：

有同学问，文中对于integration这块没有谈到，这层感觉也需要要的吧？答：integration本质就是调用外部服务实现内部的一个功能，看这个功能属于哪一层的，它可以是一个dao，也可以是一个spi实现。所以我的脑子里，integration只是infra的一类，在业务架构里不属于某一层，如图是竖列边上的，因为哪层都可以随手用。此处的分层设计，聚焦在业务架构设计层面，和一个应用的完整代码分层、包结构分层设计相比，会有简略。

实践中的细节问题

1.

api和biz层冗余了？ 不是

语义上，api层是站在应用的角度与外部应用交互约定的实现，是向外表达。biz层是应用承载业务里的某一类场景，是向内表达。虽然大多数情况下，两者是1:1的关系，且api层几乎薄到只做透传，但毕竟语义不同，也会出现N:M的情况。

举例1，业务线共用API A，然为B业务专开API B（如考虑其鉴权体系特异），因服务场景相同（如下单）以同一BIZ A表达，此刻API 和BIZ = N:1。举例2，两个服务场景，一个叫新增（如新建用户），一个叫更新（如更新用户资料），两个场景的流程、逻辑截然不同，故为两个BIZ。应用服务层面，上游调用方希望“若有则更新、若无则创建”的语义，故而是一个API（假设上游强势，且不愿自行编排）。此刻API和BIZ = 1:N。举例3，前两类同时发生，则为N:M关系。

所以个人观点是，api层并不冗余，虽然常见较薄，依然尤其独立的职责。譬如特殊业务的定制API、同一业务不同渠道（PC、APP）不同API、同一业务权限控制力度不同的API。

2.

能否跨层调用dao？ 可以

2016年在做汇金结算的重构设计时，个人的理念是，四层应有严格的次第顺序，上层只可见到直接下层，跨层是万万不可的，否则就讲不清楚架构规范了，否则就会导致调用关系混乱，否则本该沉淀在domain的逻辑因biz可直连dao而错放了位置，等等担心。

近几年，反问“为什么一定不能跨层调用？”，觉得，就算跨层调用，四层还是那四层，只要每层的职责定位清晰。反之，纵使跨层调用被禁止，落地上依然会从分层合理性的切入点腐化架构。

实践上看，“禁止跨层”会导致很多变形的动作出来。举例，query and do something，依据A的存在与否做后续逻辑的判断，这是biz层里十分常见的片段。而query往往仅是一个daoA里最简单的sql，无复杂逻辑无业务属性。在“禁止跨层”禁令下，则只能在domain里新添类似proxy的服务供biz层使用，这是所谓的“变形”，不仅大大增加了程序员的无效代码量，更使domain里充斥了无关代码（至于什么是应该放domain里的有关代码，下文在“领域设计”里会涉及）。此例仅是biz跨到dao的问题，若推想至api里需以查询daoB做鉴权之类的判断依据时，则更要凭空专造一个biz场景出来，那就越发的怪异了。

因此个人会选择放开跨层调用。本质还是四层的职责定位要清晰，过程中的架构维护要遇歪立扶，是要靠人和机制的。

3.

代码该放哪层？ 厚domain薄biz

首先问题大致出现在biz和domain这两层，因为简言之，api和dao这一顶一底是不该带业务逻辑的，故纠结点集中在中间这两层。当然，我也见过把api层业务化甚至顶替了biz层的，也见过dao层业务化之下冗长难懂的sql，但都是极少数。

首先定义下，什么叫个人认为的“复杂业务逻辑”。本文起头就提到“至少面向5类行业若干业务线且业态差异很大”，在此之下是流程、逻辑分支、模型、业务个性化的规模性爆炸，各业务线交织在庞大的逻辑空间中，从业务角度看很难看清单一业务在此空间中占据的全貌，从空间的组成单元看很难看清多少个业务依托其上牵一发而动全身，这个叫做“杂”，非沉心梳理而不得全。局部逻辑上，如周期订购类业务在订退升降下的剩余价值计算，营销类业务的优惠价格计算，结算类业务的账期账龄切账调账票款匹配，其本身呈现在PRD里就很复杂，这个叫做“烧脑”，烧的是思维逻辑的缜密性。

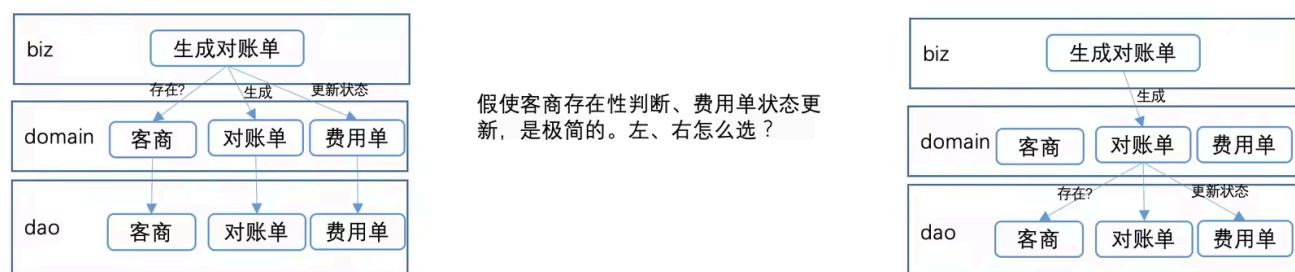
回到正题，这些复杂业务逻辑，在biz和domain里，摆放的姿势，一句话讲就是”biz编排domain service“实现业务表达。细讲开去，有2个点，第一、从系统架构内观的角度看，必须要先有domain再有biz，domain承载了该应用最核心的业务能力，好的架构里，依domain代码逐个阅读便能鸟瞰该应用的核心能力，厚domain薄biz，domain要敦实、包容、开放；第二、biz是面向场景的，核心是复用下面的domain搭建出一个业务场景，讲究灵活。biz是面向场景的一个动词视角，domain是面向一簇模型的名词视角，场景必发散，模型需收敛，动词表灵动，名词代沉稳，这是核心的差异。所以，如前文之例，切勿出现common biz这类在biz层做大复用的事情。倘若两个场景间有同性的，且不属于domain、util、infra范畴的，个人观点是宁肯做一定的代码冗余。biz管好自己是第一位，妄求复用反倒复杂化了。

4.

能否跨域调用dao？可以

首先解释下问题，即，dao何来跨域一说？由于domain是从模型间关联度的角度切割而来的，一簇紧密相关的模型确定一个domain，反过来说即一个模型归属于某个domain。而dao又是对模型存储操作的表达，推论出dao亦归属于某个域，故而domain调用dao时必有是否跨域一说。

我几次听过“不可跨域访问dao”的说法，dao只允许被所属的domain访问，跨域则生乱，如同跨层一般，乃大忌。这个论调，很容易想当然的被理解、认可和接受。然遇事之后，细想之下，便会反思。



举例，结算系统里，会有费用单、对账单、客商这些概念和域。对账单生成，这个领域服务里，需要访问客商db表check存在性（简单的pk query sql），变更上一节点模型的状态（此处是费用单）。这类跨域的操作，可以在biz层做编排，如custom.isExist(), reconBill.create(), feeBill.updateState(), 是可以实现的。但是，个人观点里，检查客户存在性、生成对账单、变更费用单状态，这三个动作，本就是“对账单生成领域服务”不可或缺的组成，因在一个领域服务里完整表达，而不仅仅是对账单

dao insert的封装。而biz层所应承担的”编排“，需在更大颗粒度上，譬如”生成对账单“并”生成应收发票“。

故，在”厚domain薄biz“的理念下，domain势必要对”隔壁域“的存储对象做些简单的无业务逻辑的操作。

澄清一点，不论是《跨层调用》里我举例的biz层下跳dao，还是此处domain的横跨dao，都仅限于”简单操作“。倘若是复杂操作，如涉及状态图控制、值枚举控制、值映射控制、对象状态变更的对外通知等业务层逻辑转换之后才能做的存储操作，是必须收口到对应的domain的。而至于简单 or 复杂的明确定义，说实话我是定义不出来的，但又不想因噎废食，所以我站在了”厚domain薄biz“架构原则这一面，而对dao合理调用控制这一面就得靠执行中的架构把控了。

再补充一个观点，上文举例”service间网状调用“使得下层呈现盘根错节的形态，不好，所以要按域垂直分桶隔离。而此处，dao不用按域分桶隔离，是因为dao之间是不会横向互调的，它是非常独立的个体。某种角度讲，dao像infra层，仅是锤子、起子、钻头这样的工具，功能单一人皆可用，只要保证使用者domain层守规矩，dao作为工具可尽量灵活。

5.

事务放哪一层？视所在业务特性而定，同一应用内需持一套标准

这个问题核心还是biz层和domain层的选择，把事务控制放在api顶层或是dao底层，我是未曾见过的。

讨论前提是，我在架构设计里一个biz层的函数调用，仅限于一个rpc同步调用的内存函数流，而非异步工作流。在此前提下，倘若biz和domain是1:1的情况（也是大多数情况），事务控制在上还是下，区别不大（因为biz本就很薄）。倘若是1:n的情况（编排若干个domain服务），放biz则有大事务的性能风险，放domain则有一致性风险，需视所在行业特性决定了。我近几年都在财务信息化研发，性能挑战相对小些，所以从架构的简约性考虑，留在biz层做事务控制。倘使有极个别协调多域成为大事务而RT不可接受的，把长流程biz拆散并以事务内消息（本地DB）进行串联，当然这会牺牲biz在语义上的完整性，个案下是个妥协性方案。而例如业务平台所面对的业务特性，譬如buy，需要编排商品、库存、营销、履约等完成一个下单，且都是rpc外部调用，即使是一个同步调用的内存函数流，也没法在biz层做事务控制，只能下放到单域服务内部，并以重试补偿等机制保证一致性。

同一个应用内部，须是同一标准，忽而在biz层控制事务，忽而在domain层控制事务，不可。

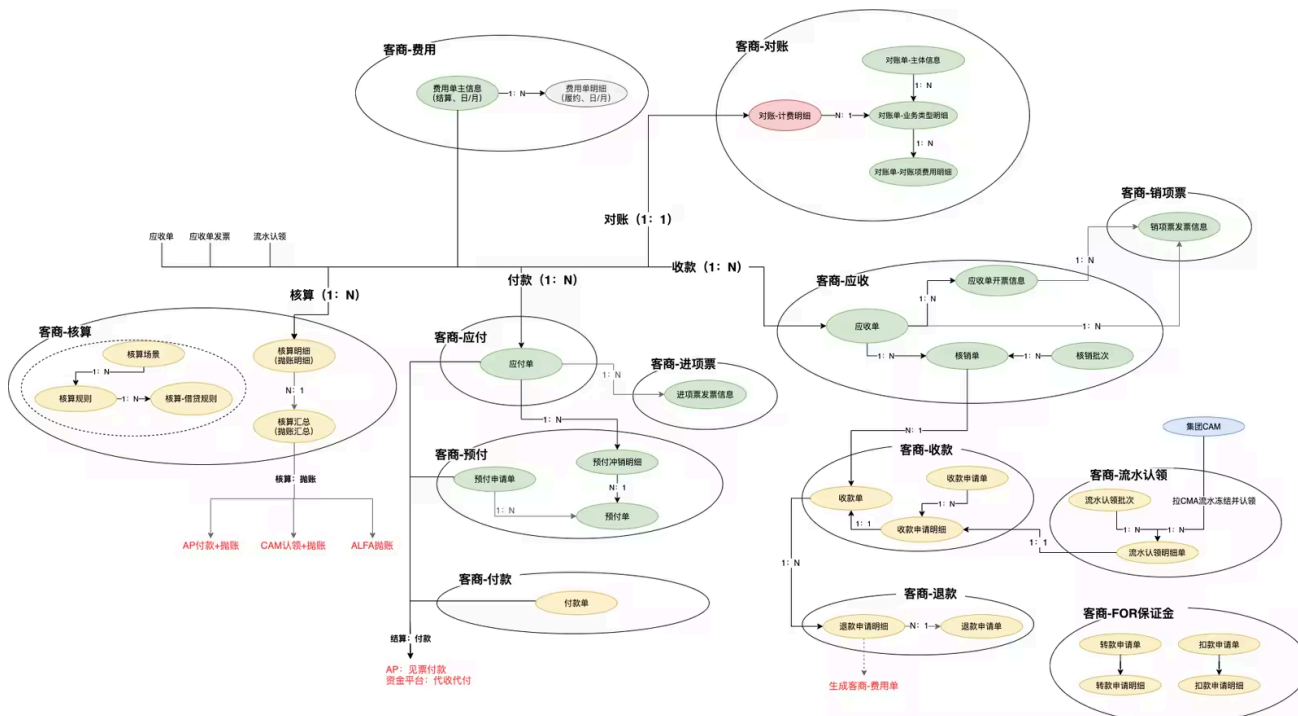
细项2 - 领域

领域设计，最核心的是划分，切的位置、切出来的大小，极大的决定了一个架构的生命长度。

个人在方法论上，最核心就两步走：第一，画全局模型图，确定网状交错的关联关系，找出耦合度最高的一簇圈为一个域，从语义的角度确定每个域的职责和服务。第二，按场景逐个画泳道图推演调用关系，检查调用的合理性、域定义的完整度和内聚度。当然，从教科理论上讲，最前面还欠缺了一步

“按user case + 鲁棒图推演出核心模型”。略过这一步是在实操层面上，这种系统性的架构设计几乎全都发生在一个业务系统已经发展了若干年，而非成立之初。对应研发同学凭经验感觉就能罗列出核心模型，且当时架构的核心矛盾不在模型完整度而在于分层分域的问题。所以这是实操上的变通简化。

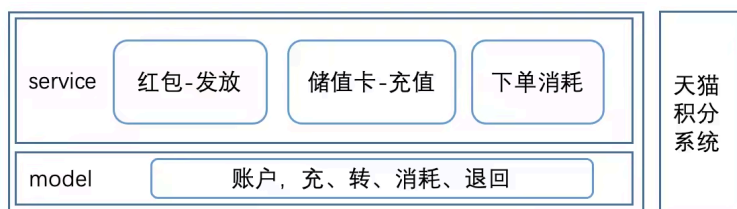
基于模型划分领域的案例，供直观感受下。近期在本地生活财务域做客商结算系统3.0的架构升级，盗用一张TL大勤画的设计图，一个圈代表一个定义出来的领域。



一个反面案例：

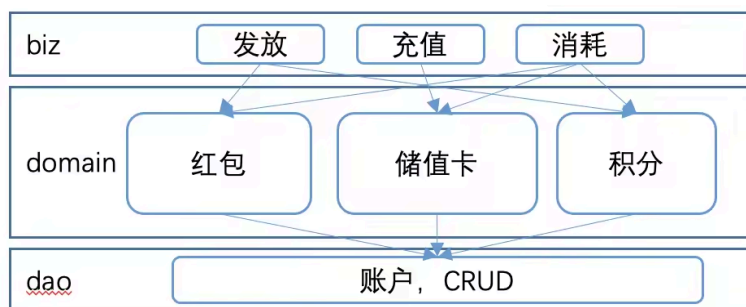
上面以模型驱动的区域切分方式，看起来似乎顺理成章，本该如此，理所当然的事情，似乎没那么复杂。这里给一个2017年自己经历的案例，走野路没摸到道前，会有纠结点的。

背景基于资金平台FP，系统可以理解为一个资金类的营销系统（如红包、储值卡、积分等等，以金本位而区别于UMP），营销的业务本来就逻辑复杂，系统也因业务拓展的速度而有腐化，还有收并的系统亟待整合（如天猫积分），故而打算做FP2.0的升级。



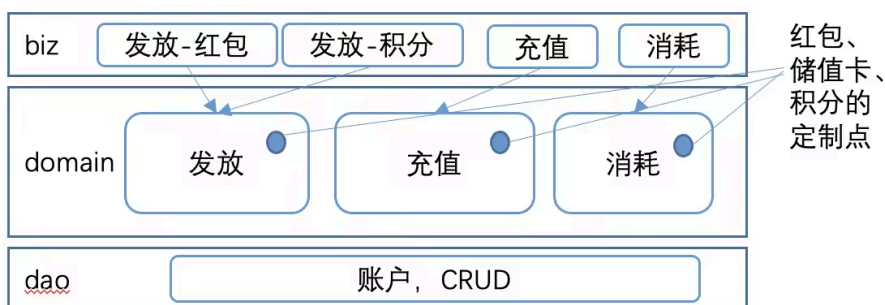
这是对当时架构形态一个极简版的表达，舍去了预算等其它域，聚焦在对账户操作这个点上，便于表达案例的关键点。第一，红包、储值卡、积分这些金本位的营销工具的表达，且属性各不相同。例如红包是平台or商家直发给c用户的，积分是随单购买返给c用户的，储值卡是用户自行开通并充值的。但在随单消耗时，又是相通的，按序或等比例作用于订单抵扣原应付金额。第二，发放、充值、消耗、退回，这些场景，是各个营销工具都可能涉及到的。第三，有系统整合的诉求，如天猫积分

谬解一，按产品划域



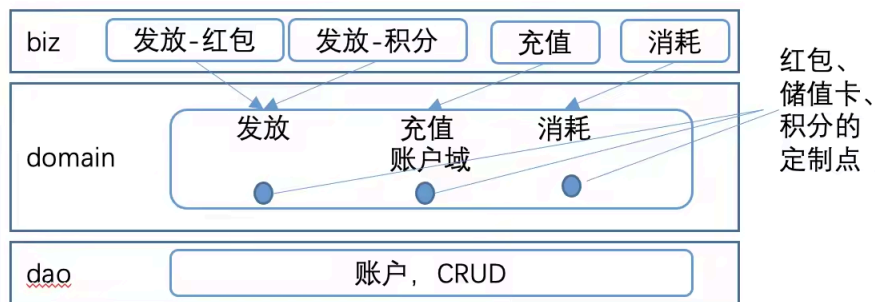
这是想到的一种解法，考虑各工具的差异性，就划分为独立的域了。当时这种想法，有个潜在的思维导向是，产品架构设计和产品表达上，就是这么归类的。这种方案下，问题有：第一，几种资金工具虽有差异，然亦有极大的相同点（如随单消耗），且不能排除差异变相同（例如在发展后期，红包亦可充值），分开表达会有很大的逻辑重复。第二，不稳定。红包、储值卡、积分，本质是产品层面的语义，随时可能发展出新的类别（譬如会员卡，即有储蓄功能、亦有会员优惠功能、还有充值送功能），也可能将现有的形态做整合。第三，性能问题，如下单抵扣这个链路，双十一是几十万tps的量级，biz的随单抵扣消耗场景，编排底下各域的领域服务进行消耗，性能上走不通。

谬解二，按场景划域



这是想到的另一种解法，这里思考的突破是，资金工具的分类仅是产品层面的语义，不能以此为系统架构设计的要素，而其差异性应以逻辑扩展的方式表达。而此处用发放、充值、消耗这些场景来切割域，思维逻辑上是认为资金工具的操作估摸也就这几个大类了，以此切分，能完整涵盖整域的表达。这种方案下，问题有：第一，biz和domain的定位更模糊了，譬如红包发放和积分发放，其流程和逻辑是不同的，故而在biz单独出两个场景。然而，这样两个流程上都全然不同的事情，就能够在发放domain里归一吗？显然会很变扭，要么就是发放domain做的很薄，逻辑都上浮到biz层，要么就是domain内部其实有两套完全独立的逻辑和流程来承载上面两个截然不同的场景biz，总之domain和biz的关系会很模糊，因为说不清“发放”这两个字到底谁负责。第二，不稳。前面用产品名切domain的问题一样，这些动词本质上是一种资金工具使用场景的表达，是产品功能层面的范畴，随时可能被演化和升级。

正解，按模型划域



这是最后采用的方案，当然仅是极简图，真实远比此复杂，还有预算域、支付域等等等等。这里的思考突破是，把所有动词、逻辑剥离，从模型上看，最后剩下的只有一个账户域。“只有一个域”，是前面的思考里一直不愿面对的，总觉得这么复杂的一件事情，怎么也要找个角度多切几刀，多整几个域来分而治之。然而从理论方法的角度看，这里涉及的模型簇就只有一个，叫做账户，所有复杂度都是操作账户前的逻辑计算（例如消耗里，一个订单按顺序or比例对各个资金工具抵扣计算、主子单的抵扣方式、某个资金工具抵扣上限控制、用户类型及商户类型和资金工具可用性的匹配度，等等等等），而这些复杂逻辑所操作的对象，都是账户，所以就是一个域而已。

那么从图上看，可能会有一个疑问，“发放”在此处标志为账户域的一个domainService，相较于上一例作为“发放域”，从biz层视角看，有啥区别呢？换言之，发放、充值、消耗，分离为单域表达，会有什么问题呢？发放、充值、消耗，都是对账户做操作，势必会有些共性的地方，譬如账户的状态图管控。状态图是带业务属性的，若冗余散落在各域必然不好，若归拢到一个叫util or infra的地方也不好（不应含业务），若下沉到dao也不好（不应含业务），若随机放到其中一个域并提供给其它域使用也不好（域间不可互调）。这类业务属性本该安置在一个领域内，但现在却无处安放，本因是把不可分割的事情分开表达了。

实践中的细节问题

1.

配置和单据同属一域吗？否

这个问题挺魔幻的，没在实操中去思考过不会觉得这是个问题。但最近客商的重构、发票的重构都有同学涉及到了里面的误区。

举个例子，上图的费用、对账、应收域里，原先都会画进一个配置模型，譬如费用单配置，表达是费用单从哪个业务来，哪类单据、哪类业务细项、聚合维度、账期日切or月切、是否需要抛账、等等等等。从某种角度说，这是个业务模型，因为它带了很多业务语义，且费用单配置、对账单配置、应收单配置，都是独立结构化表达的，而非schemaless的kv表达。再往下推演，这个模型在大图里往哪里放？既然叫费用单配置，那么就应该和费用单、费用单明细一起，组成一个费用域。逻辑似乎很顺。

然而，把费用单配置和费用单挤到一个域，带来一些变扭的地方。例如，模型关系上，费用单配置和费用单是什么关系？好像没啥关系，费用单里不会有一个字段表达是从哪个配置id来的，因为配置只是生成费用单的前置逻辑，可以是结构化配置，也可以是一段代码，跑完就完了，和目标对象费用单没直接关联关系。再譬如，聚合根是谁？应该是费用单吧，可如果这样，那似乎费用单配置并不会依赖费用单而存在吧？没有单据，也还是可以有配置先存在的。这么说，难道有两个聚合根在同一个

域？再例如，从域服务上，对上提供的域服务既要有对单据的操作服务，也要有对配置的操作服务，似乎就有两个核心对象服务了，这个和聚合根的疑问是同源的。

本质上，配置和单据本身是没关系的，它仅是表达对单据操作的一种逻辑，可以是代码或者结构化配置，但配置不是核心业务模型，不属于费用单域。

那么这个配置单模型，能否成为一个单据的域？看必要性。假使对配置的操作，更注重配置本身的信息存储功能，没有很复杂的业务逻辑，那么就由费用单域直接调用费用单配置dao读取就好了。假使对配置的操作，有丰富的控制逻辑，那就可以成为一个单独的域。例如，从结算视角看，合同也是一种配置，承载了账期、收付款方式等信息，但从合同管理本身的角度看，合同的管理亦是有着一套复杂的控制逻辑，则可以成为一个单独的域。（此处讨论的不是另外一个合同系统，而是客商系统中对于结算要用到的合同片段的信息管理）

2.

域太小怎么办？没事，如果它需要是一个域

很多次会被问到这个问题，并发生在不同系统重构的语境下。我首先会反问，这能够成为一个域吗？

不是底层有个模型依托，就可以成为一个域的。我对可成为域的定义是，对模型做CURD操作之前，需要有些业务属性的逻辑承载，否则它仅仅只是一个dao实例。

举例，客商结算系统里会有个oplog的模型，它会记录操作的单据类型、单据id、操作人、操作时间、操作内容、操作结果，会和单据操作同一事务内保存在db表里，且会有用户查看以及操作记录的合理性审计。所以它并不能完全归属到infra，它带有一点点的业务属性。然而这个模型在领域划分时，怎么都没法融入到一个具体的单据域，所以它可能是一个域。但它又不是一个域，它只是一个业务对象存储的dao，因为在对这个dao做CURD前，对此oplog的组装加工，并不需要一个单独的域服务负责。应收域在对应收单做操作时，顺便就把操作信息组装到了oplog里，然后一起保存到dao。应付域在对应付单据做操作时，也同样把应付操作相关的语义组装到了oplog里。所以这么看来，这个“对模型做CURD操作之前的业务逻辑承载”并不需要一个单独的域承载，而是散落在需要使用到oplog的地方。其实，上文提到的“配置模型”也是一样的道理，看似是一个哪儿都放置不了只能独立为一个域的业务模型，然而却并不需要一个单独的域来承载对其操作的业务逻辑。

举例，上文提到的FP资金营销系统里提到过，充值并不是一个域，只是一个使用动作和场景。但此处要补充一点，充值动作下是有个充值单来记录状态和过程信息的。那是否要单拎一个充值域来表达对充值单操作的逻辑表达呢？当时的选择是先不单拎一个域，因为当时充值这个动作仅储值卡产品有用到，不多的业务逻辑在biz层组装掉，并由biz层直接访问chargeBillDao完成记录。若发展到后期，充值这个事情变复杂了，例如可以合并充值、涉及到对充值单本身的分摊计算等，则可再行拎出为域。

所以，个人的观点是，如果它是一个域，那么再小也别和别人硬挤到一起。如果它暂时还构不成一个域，那先以dao存在着。

3.

域太大怎么办？没事，如果它只含一个聚合根

太庞大有两层意思，一是模型太多，二是代码逻辑太多。

对于模型太多，实操层面我没见过，因为一个域只能由一个聚合根，而聚合根和其附属模型间有个共生死的约定（附属不可独自苟存），所以一般以聚合根为中心的模型簇不会太大。如果有人说这个域里模型怎么这么多，一般来说应该是切的不够细了，有几簇模型组同挤一个域了。

对于代码逻辑太多，我见过挺多的。尤其是核心域，譬如FP里面的账户域，代码特别多，因为最复杂。如果以代码量为一个领域圈的大小边界，则一个系统的所有域，一个个圈圈陈列在一张大图上，呈现的景象往往是一家独大，或是双雄争霸，最多三国鼎立，很少出现诸侯割据的局面。这是正常且常见的，不要害怕一个域的代码越写越多，只要你确定它底下是单簇模型。

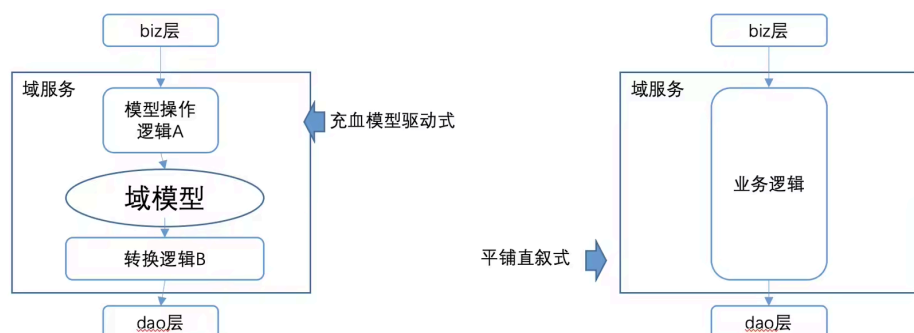
4.

它是聚合根么？看独立性

怎么看两个模型是独立的两个聚合根，还是归属到一个聚合根？个人的实操经验是，需要根据场景推演，看它是否有独立被操作、被存在的情况。切记不可以凭感觉，一定要找业务场景推演。举个例子，账户和流水，看起来是账户是聚合根，流水是附属模型，因为流水不会独立存在，流水的操作都是因账户而起。流水查询类不能算，虽然流水查询可能是独立的，譬如按用户查出所有近期流水，不论哪个账户。但是，展开去讲，从DDD里面CQRS command query responsibility segregation角度看，复杂部分更聚集在command上，所以设计考量上会更聚焦在写操作部分。所以流水是账户的附属模型，它的变化是账户金额变化的一个体现，它不独立。所以账户是聚合根，流水是附属模型，这是大多数余额类业务系统共性存在的部分，譬如资金平台、预算平台、库存系统等。但这个经验值也有例外场景。譬如，财务里有个流水认领系统，背景是to B大额交易，客户常选择银行渠道付款，常是手工打款。集团从银行账户看到的收款流水，和业务语义的应收单据比对，往往金额不一致。可能是客户合并付款、预付款，且付款备注格式不一，需要财务手工去识别、打标、拆解金额，最终核销应收单。这是集团流水认领系统的定位职责。此时，流水是独立的，它是财务操作的核心目标模型，它的操作和账户无关，故而是独立的聚合根。所以，我们需要根据场景推导，看模型的独立性来判断是否为聚合根。

5.

充血模型？不建议



对于域服务内部的实现方式，会有两种，一种我称之为“充血模型驱动式”，一种我称之为“平铺直叙式”。

DDD这本书里，一个经典的框架方式为：以模型划分域，确定该域的聚合根；以聚合根为核心模型，基于此构造若干对模型的操作函数，并以此为逻辑载体；模型操作函数，负责复杂逻辑的表达，并将处理和加工结果体现在模型实例本身的结构改变、段值改变；最后以`repository.save(模型)`的方式将其持久化至DB。

所以对域服务的实现，有些同学会基于该域的聚合根模型做逻辑操作，将结果反馈至模型实例，最后再转换为存储模型完成持久化。然而，实操层面上，事情会往杂乱的形态演化，代码和模型结构越来越复杂难懂。2016年在资金平台、2022在预算平台见过这类情况，尤其是涉及“余额”类的模型，第一感觉是很适合用此种方式表达。但事实情况是FP1.0里对红包的金额计算、预算平台里对预算池的操作，两者的代码都很复杂难懂，需要沉下心来耐心梳理方能得解。而事实上这些代码背后需要表达的逻辑又并没有代码本身表现的那么复杂，把本来略复杂的事情搞更复杂了，这是实操层面的效果。

个人理解，这里核心是两个原因。第一，以模型为中心的逻辑承载。DDD里面的例子都是极其简单的。生产实践上，尤其是面对FP这类和营销业务沾边的资金操作，预算这类和复杂树状组织架构匹配的一类资金类操作，都不是一个简单的模型能表达的。模型本身是复杂的，并且会随着业务的迭代越发复杂，呈现出父子模型、树状关系、多层级嵌套结构、稀疏态模型空间等特征。模型不能像代码那样通过SPI有很好的扩展性，不论是共性逻辑还是个性逻辑，都要在一个模型上体现，各种需求和逻辑的叠加下，模型会极限膨胀。而对于某一个域服务（对应到充血模型的一个函数方法），需要了解其逻辑，则必须要先把该模型本身整个全部摸透，就算该域服务的逻辑只涉及模型的很小一部分改动，但你还是得“沉下心来”看清楚全貌，因为你不知道这个逻辑将牵扯多少模型的结构、关系、段值的联动性改变，心慌。第二，以模型为中间载体的两段逻辑的叠加效应。DDD里的例子是简单的，所以可以模型`.operateA()`后，`repository.save(模型)`即可。现实中，模型是复杂臃肿的，将其转换回存储模型本身的逻辑代码亦会很复杂。于是在业务空间的一段需求逻辑，到了实现层面将被切割成三段，a、模型本身的理解 b、对模型基于需求翻译的操作逻辑 c、域模型到存储模型的转换。到这里可能会有个疑问，c、域模型到存储模型的转换，不是一套固定代码解决的吗，所以我们只要搞懂了a、b即可。然而事实情况是，在互联网应用里，往往不是像hibernate那种映射式save，而是mybatis那种对存储对象局部修改的实现方式，将模型里关于该域服务这一个切面的信息量，转换为sql进行持久化。因为性能。所以对每个域服务，到c阶段的代码都是不同的，还是得老老实实的看全abc才能全面知晓代码逻辑。

所以我个人是偏向于“平铺直叙”的表达方式，从业务需求到逻辑计算并直接转换为db存储的指令，从人的理解上也更容易，就算这里的局部代码是面条式，也比所谓的“充血模型”的高级设计模型，更好理解，这是个人体感上面的表达。从理论上讲，两个点。第一，平铺直叙的方式，仅针对该域服务函数的逻辑表达。典型的是query出存储对象的片段、逻辑计算加工、更新回存储对象。注意，这里的片段，仅针对当前函数，不会像充血模型式每次都捞全了。也就是一码归一码，一事一议。第二，互联网的复杂业务逻辑下，更注重的是逻辑的承载和表达。在多态业务的冲击下，流程+逻辑上的代码表达，更有利于以SPI的方式将个性化和共性化解耦，这比模型为承载体更有优势。

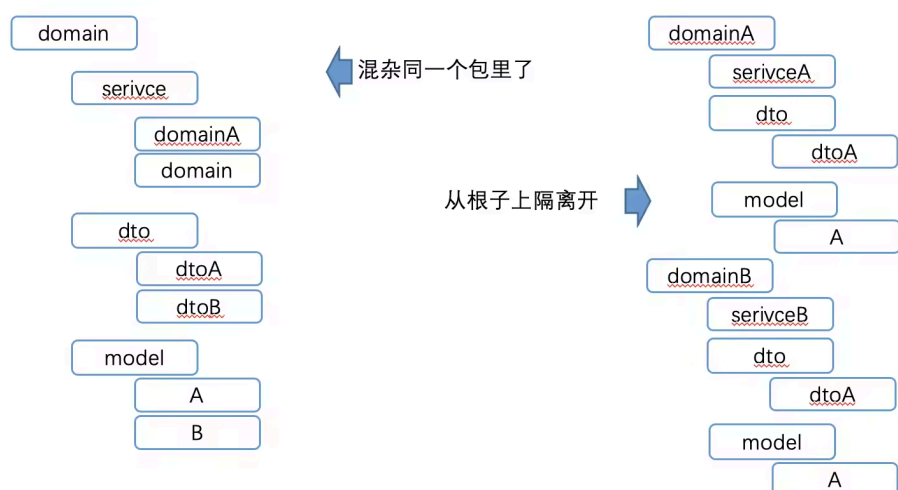
PS:

有同学问，这里的“平铺直叙”具体的实现相关。答：我核心观点是，不建议用充血模型那种以修改模型实例的状态来承载逻辑结果的做法，逻辑就用逻辑代码表达就好，不论是工厂类、责任链，在 domainService 内部内聚实现就可以。

6.

域间可以互调吗？绝对不行

这个问题显而易见，似乎都不应该是一个问题。其实我想强调的是，包结构的设计对“域间不可见”这一原则的落实性影响。



左边这种对领域包的组织方式，我见过，不好。建议采用右边这种，从根上隔开，表达的是领域和领域之间是完全独立的，就算它们内部的包分层组织类同，也请不要按分层的包名合并。隔离方式可以是 module，也可以是 package，视系统大小，但别混用。

7.

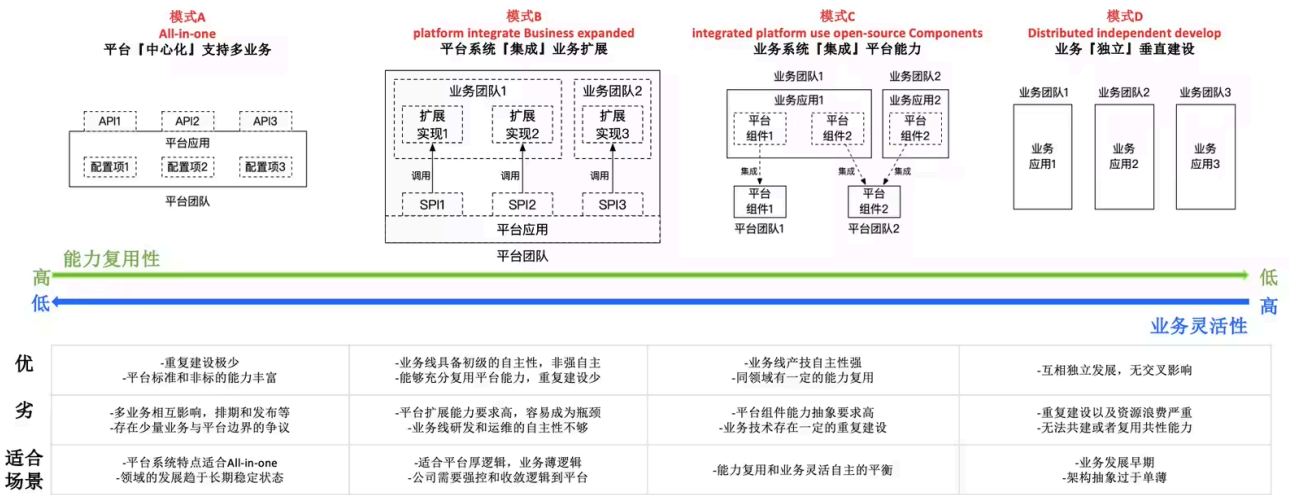
域内聚到什么程度？考虑ROI

这个问题的出发点是，曾经有同学觉得，既然领域要独立。那么，领域内部也要分 API、biz、core、model、dao 几层，因为领域们要像独立应用那样，虽然现在挤在一个应用里，但只要它们间解耦，随着业务增长和应用的庞大，我们随时可以将它们单拎成独立的应用。那么，领域内部也要做到尽量内聚，通过倒置依赖的方式（见下面《倒置依赖》有具体图例），将外界的一切实现层面的概念都屏蔽掉，让自己像个周身遍布各种型号插口的充电宝一样，只关注和知晓自己的内核，不感知不在乎将和哪个哪类型号的手机集成。

这个问题放最后阐述，因要再次强调。一般教科书上讲的“领域”，更多的是类似交易、营销、商品之于电商售卖平台的 L1 层面的领域，它们有独立的应用群、产研团队。而本文通篇提及的“领域”，仅限于 L1 域内的子域，乃至单个应用内部的域，它们共处于一个应用群、一个产研团队。”划分“这个词背后，本质是化整为零、化繁为简，在庞大的研发协同中寻求最大的效率。而在面对的是单一小组内部

的协同问题时，”解耦“的效率收益则需要和”划分“付出的代价做权衡，从实操上做一定的耦合性妥协，未尝不可。

细项3 - 开放性



<https://ata.alibaba-inc.com/articles/240175> 这篇讲菜鸟中台的文章，盗下这张图，总结的太好了，画的也好。

多态业务必有共性和个性，理想世界里，希望有一个平台将共性部分集中支撑，且同时能保持个性部分的灵动调整。现实世界里，平台集中复用和业务自主灵动，呈反向相关。实现层面，无非有四种：A 平台中心保姆式、B 平台托管SPI开放式、C 平台组件化被集成式 D行业烟囱自研式。A、D为两个方向的极端，一个极度集中复用但行业自主性极差，一个行业极度自由但无共享复用。B、C是兼顾平台集中复用和行业自主性的开放化架构方案，B以业务平台的tmf2.0架构为代表，C在“大前台小中台”技术大方向下的一种全新的行业 and 平台架构关系。

本文话题聚焦在一个应用内部对多态业务兼容性的架构表述，不涉及行业 and 平台关系的论证。然而不论是烟囱式、中心式、还是SPI开放式，被多业态业务冲击的系统，能做到“系统能力和业务个性化解耦”，“业务间解耦”，总是好的。就算整套系统就是同一方人员研发，研发内部能把业务定制代码和系统能力代码隔离开来，总是好的。就算是行业研发，也不表示面对的业务就单一了就不复杂了，这是误区。2018~2020我在集团财务研发，所谓的平台。2020至今，我在本地生活财务研发，所谓的前台行业。然而到了前台才发现，前台之前还有前台，我需要面对饿了么的餐饮、新零售、物流、城代、企餐、口碑等行业的财务需求支撑。当然，在饿了么，不会有所谓的行业财务研发来和我协同，所有的饿了么财务需求我需要全部支撑，即是平台中心保姆式。其实在人员配比合适的前提下，这种生产关系非常高效，但这并不阻碍我很需要在架构和代码层面将行业特性和系统能力区分开来的自我期望。

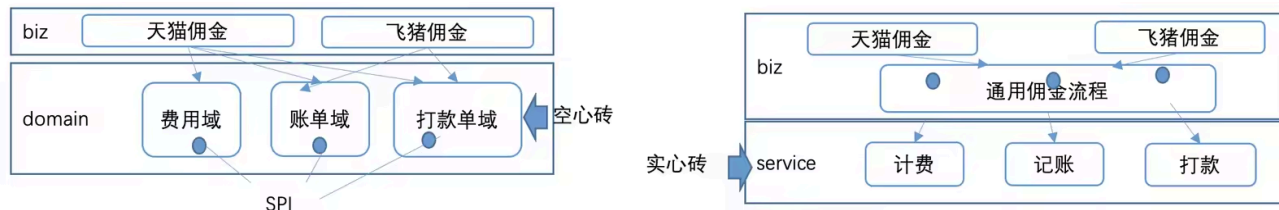
实践中的细节问题

1.

定制点开哪层？领域层

无非是在biz和domain层选择，api和dao里开定制点，我是没见过的。

按照我”厚domain薄biz“的架构思想，肯定会选择domain里开定制点。并且，从biz定位上讲，希望能灵动，针对一类场景即可，不奢求复用。倘若要开定制点，必然是在”有复用有个异“的基础上，逻辑上讲，就和biz层不求复用的宗旨矛盾了。所以必然是在domain身上动刀。



反面案例讲，原来在汇金结算经历过，见上文描述《biz层臃肿，service层单薄》这一节。原来老汇金的架构思想下，上层做编排（hjbpm，甚至有工作流式的非同步调用编排），下层提供service被编排。而下层的service没有扩展点，不可被定制，语义和内容实现上很明确，一就是一且仅只能是一，明明确确。我个人喜欢称这类底层能力叫”实心砖“。一旦这个”实心砖“的内部实现不匹配新业务形态了（就算砖头的形态、大小还是可以用的，只是局部不匹配），就得升级它。如果升级本身和其它业务冲突，则得再造一块砖。而架构上觉得这个再造同类砖是不合适的，是腐化，那就得拜托上层转换成完全匹配该砖的契合要求。于是一点点的业务差异逻辑逐渐上浮，直至biz层开始臃肿后。通用biz类和它的业务定制点就出现了。再往后发展，biz层越发的肿胀，底层service层越发的萎缩，这是一个逐渐且必然的发展趋势。

而在domain里开定制点，我个人喜好称这类底层能力叫”空心砖“，砖还是那块砖，大小、形状都没变，依然能契合原先的继承点，只是局部材质、重量、柔韧度上可通过往空心里加不同材料实现。

2.

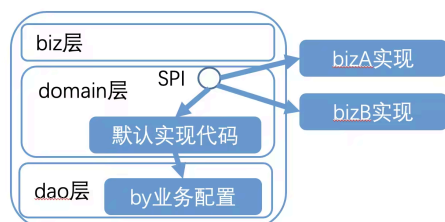
定制点开多大？不要太小

这个问题很难有个概况性的词汇来解答，本身就是个设计的”颗粒度“问题，”度“这个词本身就是一种类似”火候“这样的经验值，只能依菜而定。定制点开很大，达到几乎把所在的域服务挖空了，所有逻辑均在一个定制点里表达掉了，这个我是没见过的，估计也不会有。所以问题的核心还是”定制点要开多小？“

举个例子，18年汇金结算完成架构升级后，按业务平台组织要求，需要接星环做商业能力透出。按照当时的要求，由于要在星环对商业能力做显性化表达，包括流程、扩展点，乃至可以直接在星环运营平台配置一个商业能力的业务规则，所以要求扩展点需细，细到等同于一个配置项。以结算记账看，扩展点要细到”账期日“，”账期周期类型-月、周、日“，”账单类型-仅记账、直接结算、周期结算“，”扣款渠道-CAE、网商、资金账户“，具体收入账户id，等等（不太记得完整了），这些颗粒度正好和运营配置界面的配置项颗粒度一样，因此一个扩展点的实现逻辑直接对应到配置项值。

然而，我个人是不认同这么细颗粒度的扩展性的。因为扩展点本质上是一种业务个性化逻辑的表达，不论是代码表达，还是配置表达，甚至是一个proxy代码callback外部服务获得逻辑，而这段逻辑若是太碎，一方面不好管理，一方面不直观，一方面站在实现扩展点的研发同学的视角就更是为难了。站

在SPI实现者的视角，本来平台对其就是个黑盒，面对一堆只见其名不见其实的SPI，要把业务逻辑精准的摆放到位实为其难，更不说一堆数量庞大口径更细的SPI了。当然，站在当时设计者的角度，是希望更细的颗粒度让SPI的语义更精确。然我个人的看法是，SPI不是一个独立的个体，譬如上文提到的”账期日“，字面上确实更清晰和精确，但背后的隐忧是，我对这个SPI的实现，会联动这个平台发生什么样的变化，我是不清楚的。就像一个开关，只有on off两个选择，语义上非常的精确，但你按下去之后是关一盏灯还是关一排灯，我是不知道的。心慌。



所以，我个人的实践经验是，SPI代表一段逻辑，可以以代码表达的逻辑，之后才是两种实现方式，方式1、一段业务定制代码。方式2、一段系统默认实现代码，并读取业务配置获得定制逻辑。方式1、2是并存的，根据业务code路由实现方式。以代码逻辑打底的SPI口径不会太小。

PS:

有同学问，spi的实现路由，是biz层感知，还是domian层感知？答：是spi框架吃掉。domain只看到spi的接口。由框架来根据业务身份路由spi的实现包。biz 编排 domain，domian实现业务逻辑，并通过框架（如tmf）调用api实现业务个性化扩展

3.

业务身份定义？没标准，能横竖切开业务逻辑分而治之的，就挺好。

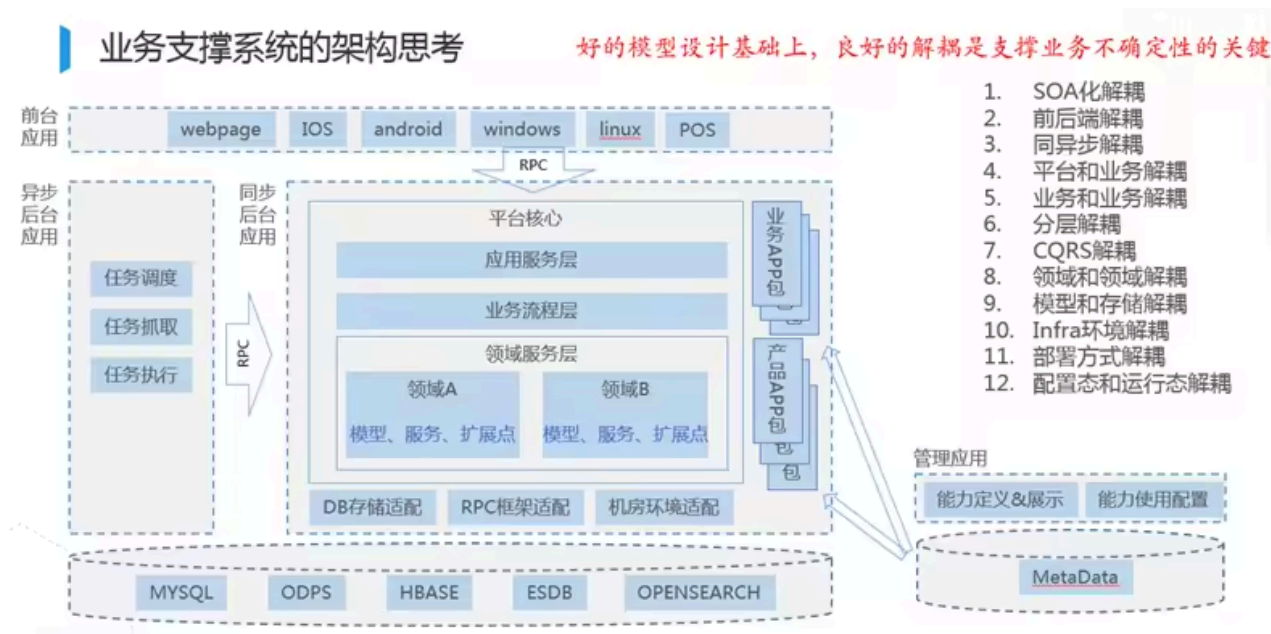
这个地方，我不用专业术语去定义一个什么叫业务，因为没法有标准方法定义。同样一个组织，看的视角不同，认出来的结果也是不同的。举个例子，CFO财务研发线，有很多对财务岗位的业务线定义，因为财务系统要支持他们的诉求，必须在某个维度将其分而治之，找出异同。然而，站在业务平台，尤其是交易、支付、营销这些toC的系统，它们会怎么看待财务岗位，是一个业务么？按照我依稀记得的2019年之前星环对业务的定义，是必须有独立own商品且有自己的商业KPI的，才叫业务。

大约2016年起，业务平台搞TMF架构，提出平台和业务隔离、业务和业务隔离、业务有业务身份的概念，我是极度认同的。尤其是有纵向业务和横向业务的区别，尤其是主架构师毗卢经常时常常拿口碑的例子普及他的架构思想（口碑是横向，淘宝是纵向，因为口碑没有自有的商品，只是一个售卖渠道），我是极以为是的。这么横竖的切分，一些问题迎刃而解。举例，在做FP2.0架构设计时，当时对于红包、储值卡、积分等产品的差异性怎么表达，很是困惑。你说它们不是身份吧，但他们确实在同一个域服务里有极大的逻辑差异，不用bizCode怎么把这个差异分支路由开呢？难不成对红包的逻辑定制，每个业务插件实现包里都写一份？你说它是业务身份吧，可是和淘宝、天猫、飞猪、盒马这些正牌业务身份，是啥关系？难不成要二元叠加为淘系-红包、飞猪-红包、盒马-储值卡。好像也挺好，记得当初真有往这个方向去想过，因为从产品形态上讲，业务和产品类别间，是一个稀疏矩阵关系。例如集团淘系共用一类红包，飞猪红包是独立的，盒马强调储值卡弱化红包。可是到后来，业务

要打通，飞猪的红包，集团也能用，但规则不同，于是出现了飞猪红包在飞猪业务下是规则A（例如使用上限之类的），在淘宝下是规则B。到这里就玩不转了。而今回看，不论这些产品类型的差异，叫做是横向业务差异也好，还是产品差异也好（TMF里也有横向产品一说），本质就同系统自身能力中有A、B、C三种模式来实现流程的同一节点，然后不同业务各自选择适合自己的模式执行该节点。只是红包、储值卡，更像是若干个节点的实现模式的一个大集合，或者叫做一个实现套餐，也是很合适的。它们本质并不是一个业务。

前面说口碑这个经典案例，引出了纵横业务的概念。这是站在交易的视角，以货权有无来定义一个业务，而当时以buy为首的交易链路视角几乎代表了整个业务平台的视角。然而，到了结算的视角，从资金往来、发票税务单元、财务核算单元、乃至经营分析单元进行业务的切分，此刻虽无货权的口碑堂堂正正的成为了一个独立的垂直业务。所以我说业务身份如何定义？因域而异

细项4 - 解耦



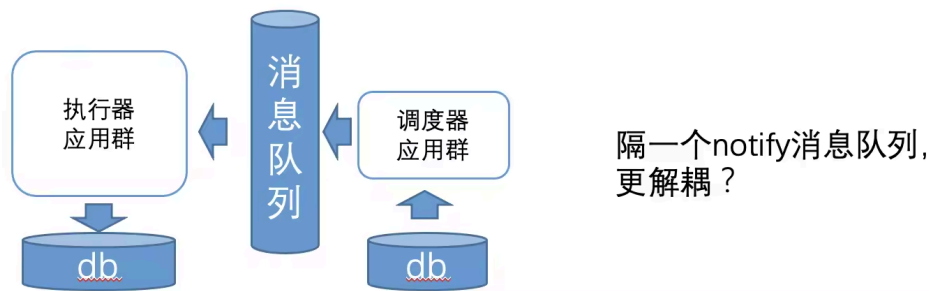
上面是一张画在2018年12月27日的图，强调如果一个系统在各方面都能做到解耦，其灵活性一定是高级的。这图到现在看，基本原则也没变，就直接拿来贴上了。

补充几个点：

1.

不是切的越碎越好

例如层次越多、领域越多、应用越多，矫枉过正了。解耦的本质是把事情分而治之，但解耦需要付出代价，收益和代价之间需根据所在业务特色、研发团队状况做权衡。当然这是句正确的废话，总之没有标准，思考事物的本质很重要。



见过一些具体的“矫枉过正”的案例。譬如图例，在调度器应用和执行器应用之间加了个消息队列做指令的传递，注意，背景是这两个应用群归属同一研发小组同一业务域。为何不以rpc同步调用方式直接传达指令？两者的qps、机器数量、稳定性保障各方面看解耦性，两种架构模式差异不大。这是一个具体的案例。再譬如应用切太细的，看似很解耦，实则因全局调用链路的复杂化而付出了更大的代价。

2.

“语义”的独立性是关键

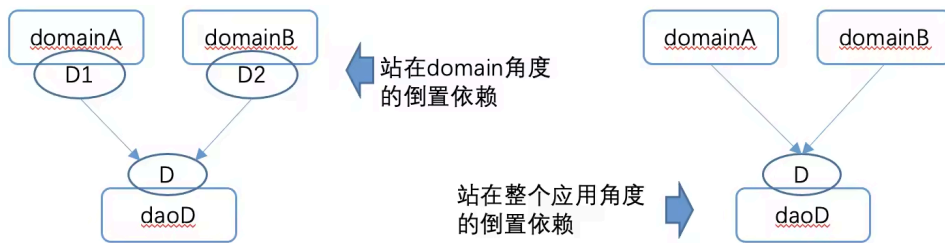
业务空间有业务空间的语义，到了技术架构空间就应该做转换、收敛。譬如预算熔断判断和预算余额查询两类业务需求，到了技术空间，可以是同一个应用服务对接。应用间也有各自的语义，譬如2017年在FP资金平台时，支付应用层面的资金组成标pmtr（我没记错的话）到了FP资金系统内部就应该转换为红包等资金工具实例的出资来源组成语义（当时却把这个pmtr的外部语义一直一直深入传递到底层，并在各层以此外部语义做逻辑判断依据，这是fp1.0代码晦涩的一个方面）。分层间也有各自的语义，在domain层叫做cancel的一个服务，到了dao层就应该理解为update(status=-1)，然而我见过上下层语义保持一个频道的，譬如biz、domain里的service有叫update的函数，看起来是做了一个很通用的函数，然而违背了本该在业务层保持语义精准性的原则。

这里扯开去一点，做业务研发，在实操中，我对关键概念名词的精准度，比较较真。不论是在研发内部的系分设计，还是对PRD的评审，都会在意一个概念的取名是否精准。因为不论是代码、还是业务逻辑的传承，都和它紧密挂钩。如果一个关键概念取名有二义性，很容易造成在协同上鸡同鸭讲，造成巨大的潜在的协同成本。往大了讲，一个业务系统架构的腐化，往往是从这些含糊其义的类型名、方法名、变量名萌芽的。取名不精准，也反映出研发同学对业务本质的思考理解不够。

3.

倒置依赖

一刀两段切开，变为调用方和服务方。站在调用方的角度，倒置依赖讲的是“我作为A，需要有服务D1供我使用”，D1这个交互协议是站在调用者需求角度提出的，是个体需求的表达。非倒置依赖讲的是“我作为A，看到有服务D适合我使用”，D是站在服务方本身具备的角度描述的，是公用资源的表达。

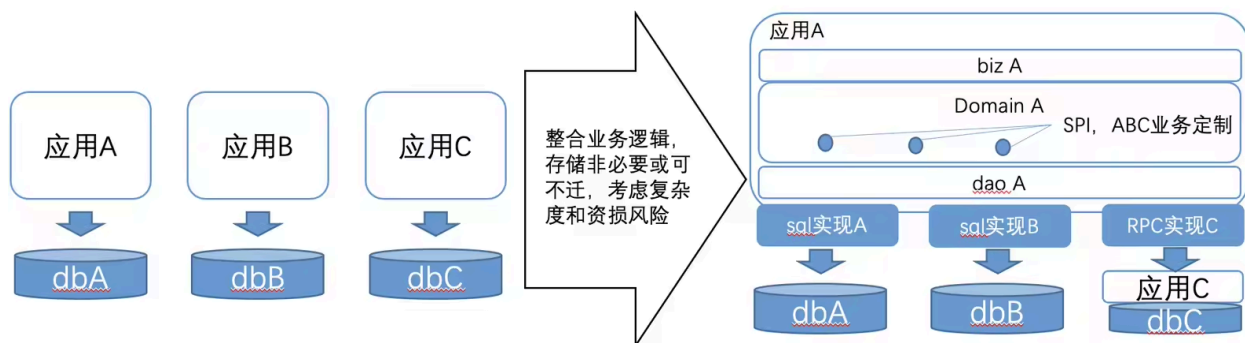


落地上有两种实现方式，第一种站在领域层级看倒置依赖，譬如域A要D1服务，域B要D2服务，而对应服务只有D，因此D1->D, D2->D是需要分别转换的，且D1、D2这个定义的交互协议，仅能在领域内部复用，作为防腐层的转换代码也应在领域内部。第二种站在应用的高度看倒置依赖，应用说我要有一个D服务放在infra层以供所有领域使用，因此A、B两个域可以共用D服务，不用做转换，甚至可以把服务协议D耦合进A、B的领域代码。而在应用层面，可以有D的不同实现，譬如针对不同表结构的dao实现、针对外部服务的rpc调用实现，可以起到对环境解耦的倒置依赖。

本质上讲，这是两种内聚颗粒度的问题。颗粒度越小，成本越高。个人会选择应用角度的倒置依赖。第一，教科书中“领域”的概念，很多是L1的，譬如交易、营销、商品这些站在销售电商平台角度讲的域，应用和团队都是要划开的。而本文的架构设计，围绕的是单域内的架构设计，可以理解为单应用内部的子域。第二，因为是单应用内部的，故而它并非真正需要那么的独立。而D1、D2这些语义的倒置依赖实现成本是巨大的，且大多数应用内划出来的域都是不大的，ROI上就要权衡了。故而，个人会选择在领域语言独立性基础上，做一定的耦合妥协，譬如此例中领域感知到infra层的协议语义D。

4.

业务模型和存储模型解耦

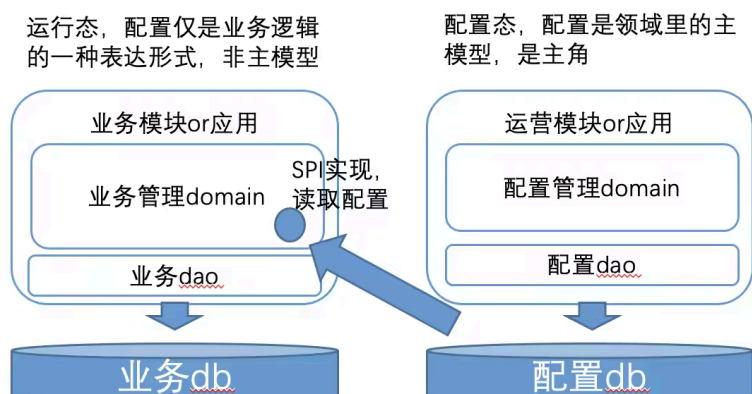


一般来说新建的系统，应用和DB都是完全掌控的，都是自家的东西，就没那么容易分清楚彼此。发展到后期，因为系统和团队的归并，需要和异构系统做整合，此时前期的业务模型和存储模型是否解耦就很关键了。举例，FP资金平台，发展到后来，需要收口集团原散落在行业的资金营销工具，例如天猫积分。此时形态，在domain层都是积分模型，到了dao层一边是自行设计的db存储模型（如飞猪旅行积分），一边是原天猫积分的db存储模型。存储层面更有甚至是http调用外部服务（当时新零售战略下，投资了些超市，它们有自己的会员储值卡系统，需要系统集成的方式对接使用）。假设前期设计的好，只是在dao层有3种不同的实现适配，核心的领域代码和模型是不用动的。

推而广之，按照DDD六边形法则，系统对外的触点，理论上都可以接口+实现的方式做环境的解耦，当然实现上也要考虑ROI。最糟糕的是，例如上面讲fp的pmtr标一样，一个外部环境的语义在整个应用中从头贯到尾，当对接的支付渠道作为外界环境发生变更时，就只能做贯穿全身的大手术了。

5.

配置态和运行态解耦



主要讲的是“配置”这个东西，读用的地方和写给的地方是两件事情。如同上文《领域设计》里提到过的案例，客商里面费用单配置，它在帮助费用单表达其生成逻辑时，这叫“运行态”，此时单据是主配置是辅。而在通过运营平台之类做配置管理时，这个叫“配置态”，此时配置是主模型，它可以成为一个域，如果足够复杂的话（譬如对配置的结构管理、配置实例的增减、多角色的编辑审批等协同）。

多系统间架构形态（反面）

多系统的组合空间巨大，没法有个标准形态。这里以个人认为的反面案例反向表达实践的总结。

1.

微服务切应用

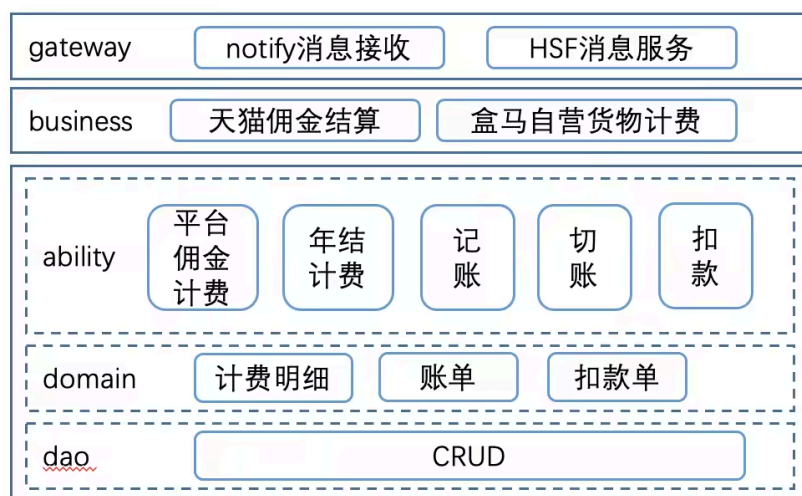
案例是15年在业务平台，支付域，由原来做国际化支付的团队合并过来的，连人带着支付应用。因为当时我在汇金的订购、结算有涉及支付，所以摸过支付的代码，切的真是碎，一个挺小的没几个类的原子的功能就可以成为一个微服务应用。

切的碎有它的设计考量，springboot让应用的建立成本大大降低，docker亦大大压缩了运维成本，从团队协同上讲把事情分的更细能尽量规避巨大的沟通成本，好处多多。

但我个人是不赞同切的那么碎的。我认同SOA，但不认同微服务，两者的区别是颗粒度问题。我认同十多年前的五彩石项目把一个巨石系统的淘宝按域切分成交易、营销、商品、结算等等，因为每个域背后是一支单独的研发小组。当组织随着业务庞大后，倘若继续守着原来的一个系统一套svn（当时行这个）是协同上的一种巨大灾难。但微服务颗粒度不同，已然是一个研发小组内部的事，还要将应用拆分到每人拿若干个应用（听说在原来AE的架构里，这里的“若干”是比较大的一个数），我不太能理解这里的好处，可能是有。但相较于应用拆分导致的长链路性能、长链路下问题排查效率、每次基础组件升级都要配合着落实N个应用，潜藏的运维成本、以及潜藏的应用交接成本、潜藏的对N个应用逐个盘摸得到全局的理解成本，远大于收益。

2.

按层切应用



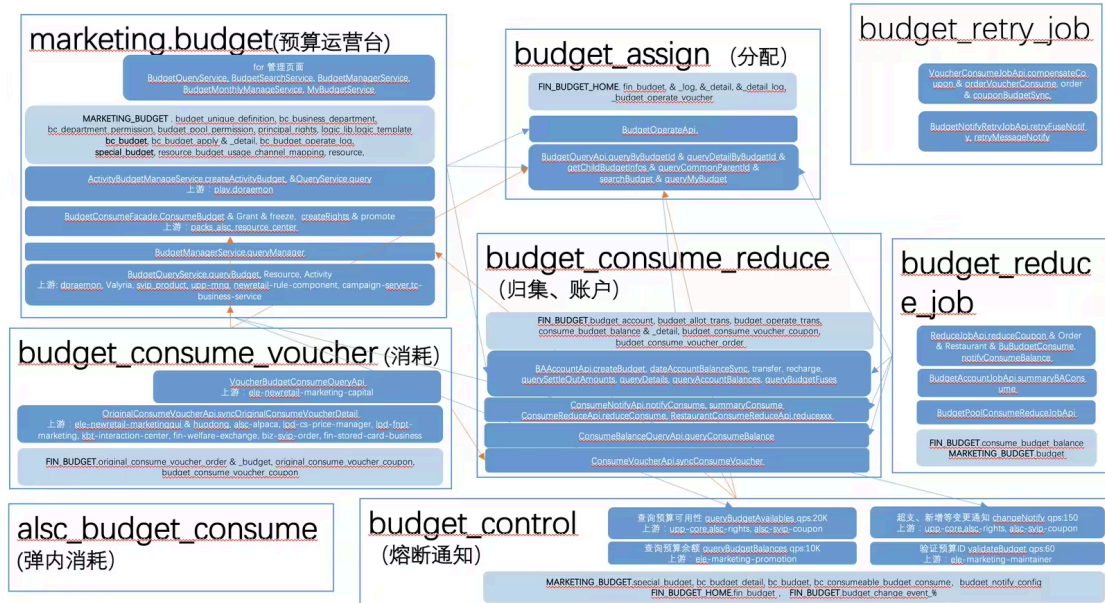
案例，16年做汇金结算的重构，设计了5个分层，网关层gateway，notify接收订单完结消息，并DB里缓存之，之后异步计费+结算；流程层business，沿用老汇金hjbpm编排底层能力，等同于标准分层架构的biz层；ability层，提供底层的原子服务能力，供上层编排；domain层，基于某个模型的操作；本质上ability层+domain层 = 标准分层架构的domain层，而划分成这两层，其实是反面的案例，实操层面就会觉得逻辑都在ability，domain几乎弱化为dao，且ability用动词划分有不稳定性问题，当然这是上文“单系统内分层架构形态”范畴的一个反面案例了，此处不展开。

回到应用划分的问题，当初受了层间腐化的苦（该沉的能力代码没沉，该浮的业务个性化在底下做判断），心有戚戚，念着重构最大的目标就是要把业务和能力分层开。又担忧现在的设计不被未来人所遵守，干错一不做二不休，直接把business, ability, domain, dao切开成单独的应用，且严格禁止跨层访问，寄期望于应用的隔离从架构上把腐化问题给杜绝了。但又碍于事务问题，ability到domain这层，一个原子ability服务会跨多个domain修改，domain的应用独立将引出跨应用调用的分布式事务问题，实难解决。为了坚持应用的独立，曾经考虑过ability到domain走二阶段事务框架，参考蚂蚁的xts分布式框架（当年很行这个，面试都会经常问二阶段问题，近几年不知为何这类问题忽然淡出了），然又碍于二阶段事务框架对domain的改造侵入之剧烈（需要domain每个方法都展开为prepare, commit, rollback三阶段，且需根据服务场景用不同的写法，写错更容易引故障，对研发要求很高），最终放弃。最终的最终，采取了折中方案，把ability以下三层打包到一个应用里，规避事务问题。虽然是折中方案，依然横切出了3个应用：hjgateway, hjbusiness, hjability。

直到2018左右支援voyage项目，因为lazada海外部署的成本要求极高，故而只能合并到一个应用做海外部署了，并借此机会把国内的应用也合并了。合并后，内部还是以不同模块做隔离，丝毫没有因此增加层间污染的风险。

3.

按领域切应用



这是2022年当下在本地生活饿了么的预算系统架构，直接拿年初梳理的草图表达了，不另画简图，更原汁原味。

饿了么的预算系统，见名可知是做预算管控的，包括对个人、商家、骑手CBD全方位营销补贴的管控。饿了么的预算系统，不仅做了前期的预算规划和数字化留存，后期的预实分析、钱效分析，更牛的是直接打通了各个上游业务系统，做到了贴近业务的准实时预算透支熔断。这个事情在单域好做（如FP的资金预算），因为做业务系统和该域的预算系统是同一拨人。但能把整个饿了么的补贴都管控起来，是很牛的，因为要主动推进对接所有的上游前台业务系统，了解前台业务，并承受业务和系统的差异性带来的接入工作量。（我21年去的时候就有这个系统了，前人的结果，所以夸的更直接些）。

但在系统设计和领域划分上，是有些问题的。

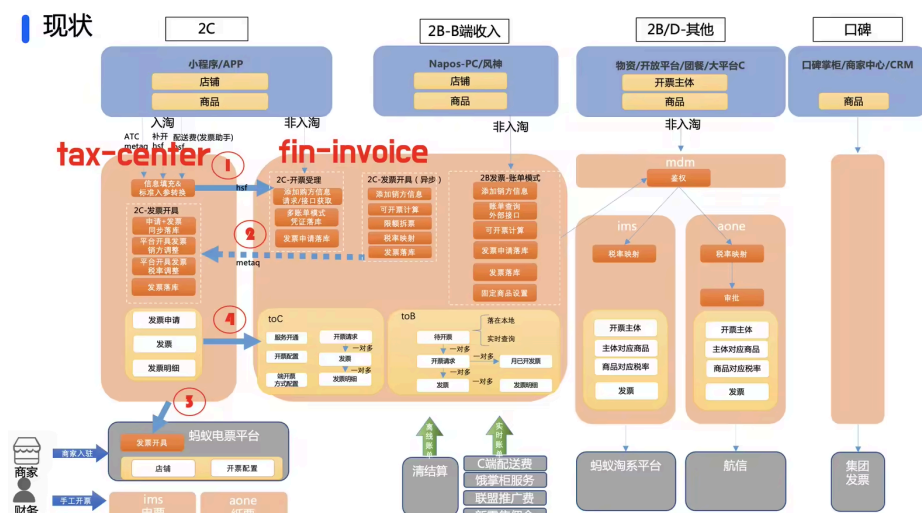
我理解当时团队是随着预算发展对业务了解的深入，做了架构演进式的领域划分，从对业务的认知经验和感觉出发，划成了动名结合体。分配、消耗、熔断是动词，账户、预算名词，按这些亦动亦名的关键概念划了单个的域，并基于此单拎为系统，db也是独立的。这是第一点，领域划分方法论的问题。全局设计上，出发点上希望对这些应用有个分层，按照预算从生产到消耗的时序，从上到下划为了分配域、消耗域、管控域。然而事实情况是这些动名词组成的应用间，呈现出上下左右互通的全网状调用态。譬如，从时序上，分配在上、消耗在中、管控在下，然后管控通知业务系统熔断活动前是需要查询预算、账户的状态、余额的，而这两个名词，却是归属在上层的分配、消耗域系统中，导致了下层调用上层。这是第二点，层次划分角度的问题。按域单拎应用，带来的巨大开支。譬如分配和账户，这在模型层面是两个模型簇，但并不代表它们就必须是两个应用。一次预算分配操作，要经历1、建立新预算模型2、将父预算下的账户余额转出3、新预算下新建账户，并接受转入的余额，极简流程大致如此。分割为两个应用，势必涉及两个应用间的调用和数据一致性问题，而这本可是一个应用里一个biz场景同一事务下对两个domain服务的编排。这是第三点，应用切分粒度的问题。

当前预算小组正在做重构的设计，目标是解决几个问题，第一，应用太散，计划会把图中的预算运营台、分配、账户、熔断四个应用合一起，把两个异步job合一起，至于消耗未合入主要是考虑这本质上是一个对消息做准实时异步聚合的事情，并非一个在线业务服务，甚至未来是可以基于实时计算技术栈承载的。第二，在合并后的应用内部做分层、分域的划分。第三、把微观设计上过于复杂导致代

码晦涩难懂的点，用更符合本质简洁易懂的设计方式再实现一遍。当然，这是另外一个维度的话题，不展开了。

4.

按环境切应用



环境差异，往小了说因为“弹内弹外”、“机房”、“技术栈”差异的区别而冗余系统，往大了说因为“国际化地域差异、业态差异”而冗余系统的。面对同一个业务域同一个语义，也经常会出现非必要的系统级别代码冗余。当然，冗余的背后，往往夹杂着“借环境差异之由行重构之实”的阳谋，但从实践经验看，往往以此为动机的结局是，新老两套系统将长期并存在两个环境中。

举一个例子，饿了么的发票系统。业务背景上，它支撑了饿了么的开票信息化，包括toC的餐费、配送费、包装费等发票，toB的佣金、广告费、配送费等发票。21年初接手的时候，是上图这个样子，不展开讲，关注带红字的地方即可。两个应用之间的故事，fin-invoice是最老的系统，负责弹外的toC+toB发票开具（所谓弹外，指的是饿了么自己的环境和技术栈，区别于2020年技术发起入淘，包括交易、营销这些应用嫁接到集团业务平台之上）。tax-center是入淘后的新应用，在弹内，接收弹内的交易消息（不仅是网络的区别，消息体也不同，因为弹内的交易系统切换至buy2）。原本应对措施是清晰的，面对“弹内”的新体系，要么是把fin-invoice再部署一套，并对新消息体做兼容，要么就是新开一个proxy接收弹内的消息，向fin-invoice的结构做适配转化，并委托其完成本该完成的开票即可（弹内弹外对应的开票流程和逻辑并无差异）。但当时的团队希望在tax-center里发展出一套全新的理想的发票系统，并逐渐取代fin-invoice老系统。于是，proxy便不仅是proxy而已了。具体来讲，步骤1，tax-center监听到“弹内”的交易消息，做消息体转换，同步调用传递给fin-invoice（此步是proxy该做的事情）步骤2，fin-invoice做拆票、税率计算等开票核心逻辑的事情，将开票请求模型保存DB，并以消息形式将开票请求异步传递回tax-center。步骤3，tax-center将收到的开票请求保存至DB，并向蚂蚁电票平台发起开票请求（这也是proxy该做的事情，因为当时弹外调蚂蚁电票不通）。步骤4，tax-center接收蚂蚁电票开票结果，保存发票记录，并更新开票请求状态。将两个模型的信息同步回fin-invoice做对等模型的同步。这个双边留存同一套数据的方案，如果只是临时性的，按团队原计划往tax-center迁移，那么这个双边性问题暂且可控。不巧的是，故事发生了变化。当时发票做了一个跨BU的交接，事情和系统给到了新BU新团队，原来负责发票的研发同学没跟着过去。于是“最终往tax-center迁移”的事情就耽搁了（新团队的想法不同）。再之后就到了21年初，连人带事再次做

了跨BU交接。这次我有参与到了，当时听到的声音是，两边的数据状态又不一致了，天天订数据不知何时是尽头。21年上半年我们投入了较大的力气，让tax-center回到了proxy本应在的职责范围。

再举一个例子，2016年在做汇金结算重构时，会强调“解耦”两个字，不仅是层层间的解耦、域域间的解耦，还要和环境解耦，不论是中间件还是DB，都希望能做到实现形态可适配而业务核心能力代码不用动。到了2017年做voyage项目时，lazada在东南亚机房的技术栈大不相同（中间件），乃至db存储模型也不同（时区、多租户的差异），我们并没有重新搞一套系统和代码，仅实现了国际化适配层，最核心的business、ability、domain代码没有大动，相对比较高效快速的支撑了voyage项目。